Edge cases (empty vs. missing) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Where this exercise is saved: `practice/env_config_practice.py`, under its `# Edge cases` section — the same file also holds the Intermediate exercise, under an `# Intermediate` section. Run it with `cd practice && python env_config_practice.py`.

Builds on: your practice file of that same file — you are testing *that* loader against the two tricky `.env` states, so keep both sections in the one file. The `MissingConfigError` used below is the same class the Build Task ends up with in `practice/build_task/exceptions.py`.

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint builds toward the full solution.

- Hint 1 — The idea, and try it yourself
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and try it yourself

Python's `os.getenv()` already treats these two cases differently, on its own — spotting that is the whole exercise:

```
# practice/env_config_practice.py — Edge cases section
os.getenv("OPENAI_API_KEY")   # key never set at all       -> returns None
os.getenv("OPENAI_API_KEY")   # file has "OPENAI_API_KEY="  -> returns ""
```

`None` and `""` are two different values in Python. `None` means "this key was never set." `""` (an empty string) means "this key was set, but to nothing." Your `require_env()` has to decide whether *both* of those should raise `MissingConfigError`, or only the first.

Test both situations against your own `require_env()` (or equivalent) from the Build Task, and inspect the value precisely instead of guessing:

```
# practice/env_config_practice.py — Edge cases section
value = os.getenv("OPENAI_API_KEY")
print(repr(value))   # None -> prints: None
                      # ""   -> prints: ''
print(type(value))   # <class 'NoneType'>  or  <class 'str'>
```

Using `repr()` (instead of a plain `print(value)`) shows you the quotes around a string, so you can't mistake an empty string `''` for anything else.

Hint 2 — The plan, and almost the whole thing

```
test 1: empty .env file
    run load_config()
    observe: does it raise MissingConfigError, or does it silently continue?
    write down which one happened, and whether that's what you want

test 2: .env contains "OPENAI_API_KEY=" (empty value)
    run load_config()
    observe: is the value None, or an empty string ""?
    decide: should an empty string count as "missing"?
    write down your answer and why
```

If your current `require_env()` only checks `if value is None`, an empty string will slip through as "valid" — is that the behavior you actually want for a secret like an API key?

```
# practice/env_config_practice.py — Edge cases section
def require_env(key: str) -> str:
    value = os.getenv(key)
    if value is None or value == "":
        raise MissingConfigError(
            f"Required environment variable is missing: {key}"
        )
    return value
```

What's missing: actual test calls, and the lenient version (which only checks `value is None`, with no `== ""` part). Write both yourself, and try them against an empty `.env` file and a `.env` with `OPENAI_API_KEY=`, then compare against the Solution.

Full solution: Show me the solution